

Eléments de logique pour l'informatique

Uli Fahrenberg
`uli@lmf.cnrs.fr`

Département Informatique, Faculté des Sciences d'Orsay, Université Paris-Saclay

Licence Informatique & LDD Informatique, Mathématiques

2026–27

Introduction au cours de logique

- 1 Motivations
- 2 Organisation du cours
- 3 Exemples d'usage de la logique

La logique, chemin vers la vérité

- *Logique* vient du grec logos (raison, langage, raisonnement).
- La logique manipule des *énoncés* (phrases dont le sens est vrai ou faux).
 - “Tous les moutons ont 5 pattes”
 - “Aucun étudiant ne joue sur son téléphone pendant le cours de logique”
 - “Les trains ne passent pas à un passage à niveau ouvert”
- “La couleur du cheval blanc d’Henri IV” *n’est pas un énoncé*
- Importance du langage pour préciser de quoi on parle (souvent implicite)
- Un énoncé peut être vrai ou faux suivant la manière dont on *interprète* les mots
- La logique est une manière *scientifique* d’étudier la notion de vérité

Combinaison d'énoncés

- L'arithmétique étudie les propriétés des nombres : $0, 1, 2, 3, \dots$
- La logique (classique) s'intéresse à un espace beaucoup plus simple : *vrai / faux*
- Les **connecteurs logiques** sont des *opérations* qui permettent de former de nouveaux énoncés
 - la **négation** d'un énoncé E : la négation d'un énoncé E est vraie si et seulement si l'énoncé E est faux
 - la **conjonction** : l'énoncé E_1 *et* E_2 est vrai si et seulement si les énoncés E_1 et E_2 sont simultanément vrais
 - la **disjonction** : l'énoncé E_1 *ou* E_2 est vrai si et seulement si l'un des deux énoncés E_1 et E_2 est vrai (ou les deux)
 - ...

Énoncé paramétré et quantification

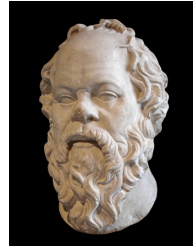
- Un énoncé logique *paramétré* représente une propriété (vraie ou fausse) d'un "*objet*" quelconque
- exemple : propriété pour un étudiant de *valider le cours de logique*
- "*valider le cours de logique*" sera vrai ou faux pour chaque étudiant considéré
 - *Toto valide le cours de logique*
- on peut former de nouveaux énoncés comme :
 - tous les objets vérifient la propriété,
 - il existe (au moins) un objet qui vérifie la propriété,
 - il existe précisément 42 objets qui vérifient la propriété.

La logique, chemin vers la vérité

- Le **raisonnement** est un cheminement (une **déduction**, une **preuve**) qui permet de relier entre eux des énoncés : on distingue des *hypothèses*, et une *conclusion*.
- On veut s'assurer que dans toute situation où les *hypothèses sont vraies*, il en est de même de la conclusion.
- Un raisonnement suit des *règles* logiques précises.
- Le raisonnement est un procédé suffisamment élémentaire pour *convaincre*
 - modus ponens, preuve par l'absurde, preuve par récurrence ...
- **Montrer** qu'un énoncé est vrai est, en général, **indécidable** (il n'y a pas de programme qui répond à cette question).
- **Vérifier** qu'un raisonnement suit bien les règles du jeu peut, le plus souvent, s'effectuer mécaniquement.
- Certains raisonnements sont adaptés à l'humain, d'autres aux machines.

Exemples de raisonnement

*Tous les hommes sont mortels
or Socrate est un homme
donc Socrate est mortel.*



*Tous les hommes sont mortels
or l'âne Francis n'est pas un homme
donc l'âne Francis est immortel.*



Dialogue entre le Logicien et le Vieux Monsieur, extrait de Rhinoceros (Ionesco)

- Je vais vous expliquer le syllogisme.
- Ah ! oui, le syllogisme !
- Le syllogisme comprend la proposition principale, la secondaire et la conclusion.
- Quelle conclusion ?
- Voici donc un syllogisme exemplaire. Le chat a quatre pattes. Isidore et Fricot ont chacun quatre pattes. Donc Isidore et Fricot sont chats.
- Mon chien aussi a quatre pattes.
- Alors, c'est un chat.
- Donc, logiquement, mon chien serait un chat.
- Logiquement, oui. Mais le contraire est aussi vrai.
- C'est très beau, la logique.
- A condition de ne pas en abuser...

Démonstration

- Démontrer c'est apporter une *évidence* du fait que quelque chose est vrai

- Plusieurs sortes de preuve 😊

(<http://www.pion.ch/Logic/preuves.html>)

- par l'exemple : *On démontre le cas $n = 2$ qui contient la plupart des idées de la preuve générale.*
- par généralisation : *Ça marche pour 17, donc ça marche pour tout nombre réel.*
- par fin de cours : *Vue l'heure, je laisserai la preuve de ce théorème en exercice.*
- par probabilité : *Une recherche longue et minutieuse n'a mis à jour aucun contre-exemple.*
- par tautologie : *Le théorème est vrai car le théorème est vrai.*

- Dans ce cours 😐

- des méthodes rigoureuses
- transposables sur ordinateur

- La logique *formalise un langage*, en *définit le sens* et propose *des règles du jeu* qui permettent de se convaincre de la vérité d'une argumentation ou au contraire de la réfuter.
- *Questions logiques :*
 - avec quels *objets* joue-t-on ? que représentent-ils ? quelles règles ?
 - les règles du jeu sont-elles trop *laxistes* (on déduit des choses fausses, on dit que le système est *incohérent*)
 - les règles du jeu sont-elles trop *strictes* (on n'arrive pas à prouver quelque chose qui est pourtant correct, on dit que le système est *incomplet*)
 - peut-on changer les règles du jeu ? changer de style ?
- *Questions informatiques :*
 - un ordinateur peut-il *raisonner* ?
 - est-ce qu'il existe un *algorithme* pour dire qu'une formule est vraie, est fausse ?
 - étant données des hypothèses et une conclusion, peut-on reconstruire une déduction ?

- Le questionnement sur les fondements des mathématiques date du début du 20ème siècle avec la théorie des ensembles
- Quelques logiciens importants : Tarski, Russell, Hilbert, Gödel ...
- Des surprises :
 - le raisonnement ne peut pas se ramener au calcul
 - impact du langage sur la cohérence : paradoxe de Russell
 - tout est ensemble, $x \in X$, compréhension $\{x|P(x)\}$
 - $X = \{x|x \notin x\}$, $X \in X$ si et seulement si $X \notin X$
 - il n'y a pas d'ensemble de tous les ensembles/il faut structurer les ensembles avec des *types*
 - théorème d'incomplétude de Gödel :
 - soit un système de déduction dans lequel on peut raisonner sur les entiers
 - il existe une formule *C* telle que on ne peut démontrer ni *C*, ni la négation de *C*
 - aucun système de démonstration "puissant" (ex. arithmétique, théorie des ensembles) ne capture toute la vérité...

- *Logique mathématique* : les énoncés mathématiques, le raisonnement sont l'objet de l'étude comme les nombres en algèbre, les fonctions en analyse, les espaces vectoriels . . .
- On raisonne sur ces objets, on établit des théorèmes :
 - *deux niveaux* différents qu'il ne faut pas confondre.
- Analogie informatique : programmes qui manipulent d'autres programmes (*les compilateurs*)

- Lien entre calcul booléen (vrai/faux) et circuits logiques (0/1)
- Une démarche analogue : machine universelle reposant sur un nombre limité d'opérations ; liens entre syntaxe et sémantique.
- Intelligence artificielle : munir un ordinateur qui sait calculer de capacité de raisonnement nécessite de transformer le raisonnement en calcul (méthodes symboliques liées à la logique, enjeu de l'explication des méthodes statistiques).
- Outil de modélisation : contraintes dans les bases de données, développement de programmes, web sémantique, apprentissage symbolique, ... (logique au service de l'informatique)
- Structures informatiques pour représenter des propriétés logiques, outils pour les manipuler (informatique au service de la logique).

Introduction au cours de logique

- 1 Motivations
- 2 Organisation du cours
- 3 Exemples d'usage de la logique

- Prérequis
 - les bases du calcul booléen
 - compréhension des formules et preuves mathématiques élémentaires
 - Connaître et savoir manipuler le langage de la logique du premier ordre
 - savoir traduire des formules logiques en langue naturelle
 - savoir modéliser un problème en termes logiques
 - reconnaître certaines catégories de formules
 - savoir donner un sens aux formules de la logique
 - Savoir mettre en œuvre plusieurs notions de démonstration
 - Connaître les principales limites des méthodes d'un point de vue calcul
- LDD** Connaître et comprendre quelques résultats clés de la logique : complétude, compacité, démonstrations. . .

- Savoir présenter un raisonnement scientifiquement correct
- Apprendre un nouveau langage formel
- Distinguer syntaxe et sémantique
- Savoir manipuler des algorithmes sur des objets symboliques
- Méthodologie : mise en pratique d'objets mathématiques utiles en informatique

Plan du cours

- 1 Maîtriser le langage logique
- 2 Donner du sens aux formules
- 3 Manipuler les formules de la logique
- 4 Automatiser les démonstrations

Bibliographie

- Espace ecampus des formations
 - emplois du temps, groupes, examens. . .
- Espace ecampus du cours *Éléments de logique pour l'informatique*
 - Espace partagé entre les parcours licence et LDD
 - Notes de cours disponibles (distribuées)
 - Exercices pour le contrôle continu (à venir)
 - Annales des partiels-examens des années précédentes (beaucoup de corrigés)

- Deux cours cette semaine
- Démarrage des TD la semaine prochaine
- LDD IM+magistère : complément de cours + exercices
- Evaluation
 - Partiel (40%) + Examen final (50%)
 - un exercice sous forme *QCM* (contrôle notions élémentaires)
 - CC (10%) exercices en ligne, devoir
- Des tests à compléter sur ecampus (respecter les dates)
 - Enquête rentrée sur la page d'accueil
 - Tests (calcul propositionnel) dans la section *Maîtriser le langage logique*

- Uli Fahrenberg
 - responsable cours, chargé TD groupe 6
 - aussi responsable Magistères
 - `uli@lmf.cnrs.fr`
- Safia Tehami
 - chargée TD groupe 5
- Aquilina Al Khoury
 - chargée TD groupe 4
- Jérémie Marrez
 - chargé TD groupe 3
- Gérald Forhan
 - chargé TD groupe 1

Introduction au cours de logique

1 Motivations

2 Organisation du cours

3 Exemples d'usage de la logique

- Résoudre des problèmes
- Modéliser, prouver

Jeu du Sudoku

8		4				2		9
		9				1		
1			3		2			7
	5		1		4		8	
				3				
	1		7		9		2	
5			4		3			8
		3				4		
4		6				3		1

Règles du jeu :

- les chiffres de 1 à 9 apparaissent une et une seule fois sur chaque ligne, chaque colonne et dans chaque cadran 3×3

Jeu du Sudoku

8		4				2		9
		9				1		
1			3		2			7
	5		1		4		8	
				3				
	1		7		9		2	
5			4		3			8
		3				4		
4		6				3		1

Règles du jeu :

- les chiffres de 1 à 9 apparaissent une et une seule fois sur chaque ligne, chaque colonne et dans chaque cadran 3×3

Solution :

8	7	4	6	5	1	2	3	9
2	3	9	8	4	7	1	6	5
1	6	5	3	9	2	8	4	7
6	5	7	1	2	4	9	8	3
9	4	2	5	3	8	7	1	6
3	1	8	7	6	9	5	2	4
5	2	1	4	7	3	6	9	8
7	8	3	9	1	6	4	5	2
4	9	6	2	8	5	3	7	1

Modélisation propositionnelle

- variables à valeur vrai/faux
- position $p = (i, j)$ sur la ligne i et la colonne j
- variable propositionnelle x_p^k : vraie si le chiffre k est à la position p
- au total $9 \times 9 \times 9 = 729$ variables, soit $2^{729} \simeq 10^{219}$ possibilités
- exemples de règles du jeu
 - à généraliser pour chacune des 81 positions

$$x_{1,1}^1 \vee x_{1,1}^2 \vee x_{1,1}^3 \vee x_{1,1}^4 \vee x_{1,1}^5 \vee x_{1,1}^6 \vee x_{1,1}^7 \vee x_{1,1}^8 \vee x_{1,1}^9$$

- à généraliser pour chacune des 81 positions et chacune des 9 valeurs possibles

$$x_{1,1}^1 \Rightarrow (\neg x_{1,2}^1 \wedge \neg x_{1,3}^1 \wedge \neg x_{1,4}^1 \wedge \neg x_{1,5}^1 \wedge \neg x_{1,6}^1 \wedge \neg x_{1,7}^1 \wedge \neg x_{1,8}^1 \wedge \neg x_{1,9}^1)$$

- ...

- grille initiale : force certaines variables à vrai

$$x_{1,1}^8 \quad x_{1,3}^4 \quad x_{1,7}^2 \dots$$

Exemple, complet (?)

8		4				2		9
		9				1		
1			3		2			7
	5		1		4		8	
				3				
	1		7		9		2	
5			4		3			8
		3				4		
4		6				3		1

$$(x_{1,1}^8 \wedge x_{1,3}^4 \wedge \dots \wedge x_{2,3}^9 \wedge \dots \wedge x_{9,9}^1) \wedge$$

$$(x_{1,1}^1 \Rightarrow (\neg x_{1,2}^1 \wedge \neg x_{1,3}^1 \wedge \neg x_{1,4}^1 \wedge \neg x_{1,5}^1 \wedge \neg x_{1,6}^1 \wedge \neg x_{1,7}^1 \wedge \neg x_{1,8}^1 \wedge \neg x_{1,9}^1)) \wedge$$

$$(x_{1,1}^2 \Rightarrow (\neg x_{1,2}^2 \wedge \neg x_{1,3}^2 \wedge \neg x_{1,4}^2 \wedge \neg x_{1,5}^2 \wedge \neg x_{1,6}^2 \wedge \neg x_{1,7}^2 \wedge \neg x_{1,8}^2 \wedge \neg x_{1,9}^2)) \wedge \dots$$

$$(x_{9,9}^9 \Rightarrow (\neg x_{9,1}^9 \wedge \neg x_{9,2}^9 \wedge \neg x_{9,3}^9 \wedge \neg x_{9,4}^9 \wedge \neg x_{9,5}^9 \wedge \neg x_{9,6}^9 \wedge \neg x_{9,7}^9 \wedge \neg x_{9,8}^9)) \wedge$$

$$(x_{1,1}^1 \Rightarrow (\neg x_{2,1}^1 \wedge \neg x_{3,1}^1 \wedge \neg x_{4,1}^1 \wedge \neg x_{5,1}^1 \wedge \neg x_{6,1}^1 \wedge \neg x_{7,1}^1 \wedge \neg x_{8,1}^1 \wedge \neg x_{9,1}^1)) \wedge \dots$$

$$(x_{1,1}^1 \Rightarrow (\neg x_{1,2}^1 \wedge \neg x_{1,3}^1 \wedge \neg x_{2,1}^1 \wedge \neg x_{2,2}^1 \wedge \neg x_{2,3}^1 \wedge \neg x_{3,1}^1 \wedge \neg x_{3,2}^1 \wedge \neg x_{3,3}^1)) \wedge \dots$$

$$(x_{1,1}^1 \vee x_{1,1}^2 \vee x_{1,1}^3 \vee x_{1,1}^4 \vee x_{1,1}^5 \vee x_{1,1}^6 \vee x_{1,1}^7 \vee x_{1,1}^8 \vee x_{1,1}^9) \wedge \dots$$

$$(x_{9,9}^1 \vee x_{9,9}^2 \vee x_{9,9}^3 \vee x_{9,9}^4 \vee x_{9,9}^5 \vee x_{9,9}^6 \vee x_{9,9}^7 \vee x_{9,9}^8 \vee x_{9,9}^9)$$

Trouver une solution propositionnelle

- les formules peuvent être “simplifiées”

- Ensemble de **clauses** (disjonctions)

$$\neg x_{1,1}^1 \vee \neg x_{1,2}^1, \neg x_{1,1}^1 \vee \neg x_{1,3}^1, \neg x_{1,1}^1 \vee \neg x_{1,4}^1, \dots$$

- au total : 10287 clauses
- propager les variables résolues pour simplifier les clauses et résoudre plus de variables
- si $x_{1,1}^8$ est vraie, alors
 - $\neg x_{1,1}^8 \vee \neg x_{1,2}^8$ devient $\neg x_{1,2}^8$ qui sera propagé
 - $x_{1,2}^1 \vee x_{1,2}^2 \vee x_{1,2}^3 \vee x_{1,2}^4 \vee x_{1,2}^5 \vee x_{1,2}^6 \vee x_{1,2}^7 \vee x_{1,2}^8 \vee x_{1,2}^9$ devient $x_{1,2}^1 \vee x_{1,2}^2 \vee x_{1,2}^3 \vee x_{1,2}^4 \vee x_{1,2}^5 \vee x_{1,2}^6 \vee x_{1,2}^7 \vee x_{1,2}^9$
 - $\neg x_{1,2}^8 \vee \neg x_{1,3}^8$ disparaît car toujours vrai
- Si toutes les clauses restantes ont au moins 2 variables alors on explore les deux possibilités pour une variable : vraie ou fausse
- Si on tombe sur une contradiction (clause fausse = règle du jeu non respectée), alors on revient en arrière pour explorer une autre branche.

- Modélisation : programme qui engendre les clauses du Sudoku
- Recherche de solution : procédure DPLL¹ qui cherche des valeurs de variables qui rendent vraies un ensemble de clauses (SAT : satisfiabilité)
- Mise en oeuvre en Ocaml :
 - 170 lignes pour les clauses et la procédure SAT (une solution ou toutes les solutions, sans optimisation)
 - 150 lignes pour la modélisation du Sudoku générique en la dimension

Introduction au cours de logique

1 Motivations

2 Organisation du cours

3 Exemples d'usage de la logique

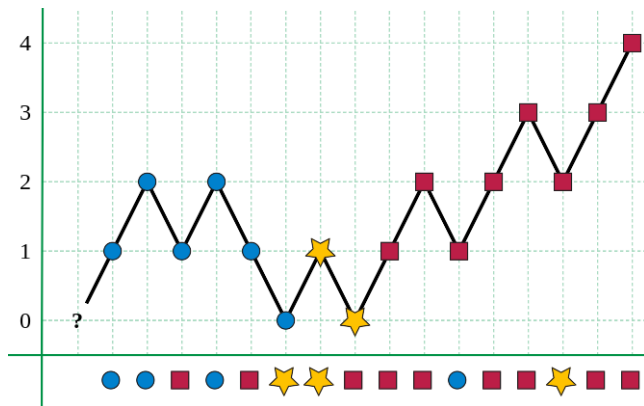
- Résoudre des problèmes
- **Modéliser, prouver**

- une **modélisation propositionnelle** se résout par calcul mais est peu naturelle et ne couvre que des propriétés *finies*
- la **logique des prédicats** permet de raisonner sur des ensembles potentiellement infinis d'objets, de manière plus concise

Preuve de programmes : Algorithme de vote majoritaire de Boyer-Moore pour chercher une valeur *m* ayant possiblement la majorité absolue dans un tableau *t* avec un seul compteur *c*

```
let majority t =  
  let rec fmaj i c m =  
    if i = Array.length t then m  
    else let x = t.(i) in  
      if c = 0 || m = x then fmaj (i+1) (c+1) x  
      else fmaj (i+1) (c-1) m  
  in fmaj 0 0 t.(0)
```

Principe de l'algorithme



- examen séquentiel des valeurs (i de 0 à $\text{length } t - 1$)
- m est le candidat majoritaire potentiel (aucun autre n'a la majorité)
- deux valeurs différentes $m \neq x$ s'annulent mutuellement
- il y a c occurrences de m qui n'ont pas été annulées

- Propriété attendue :
 - aucune autre valeur que le résultat a la majorité absolue dans t
 - $x \neq m$ apparaît au plus $(\text{length } t)/2$ fois dans t
- Invariant à l'étape i, c, m :
 - $x \neq m$ apparaît au plus $(i - c)/2$ fois parmi les i premiers éléments de t
 - m apparaît au plus $\frac{i-c}{2} + c$ fois parmi les i premiers éléments de t
- Terminaison : $(\text{length } t) - i$ décroît strictement en restant positif
- Théorie utilisée :
 - arithmétique (linéaire)
 - tableau (en lecture)
 - spécifique : nombre d'apparitions d'une valeur dans un segment de tableau

Preuve de programme en Why3 : théorie logique

```
use int.Int use array.Array
```

```
(* nombre d'occurrences de x dans les i premiers éléments de t *)
```

```
function nbc (x:int) (t:array int) (i : int) : int
```

```
axiom nbc0 : forall x t. nbc x t 0 = 0
```

```
axiom nbceq : forall x t i.
```

```
  0<=i<length t -> t[i]=x -> nbc x t (i+1) = 1 + nbc x t i
```

```
axiom nbcneq : forall x:int. forall t i.
```

```
  0<=i<length t -> t[i]<>x -> nbc x t (i+1) = nbc x t i
```

```
function nb (x:int) (t:array int) : int = nbc x t (length t)
```

```
(* majorité absolue *)
```

```
predicate maj (m :int) (t:array int) = length t < 2 * nb m t
```

```
(* invariant de l'algorithme *)
```

```
predicate inv (t : array int) (i : int) (c : int) (m : int)
```

```
  = 0<=c /\ 0<=i<length t
```

```
  /\ 2 * nbc m t i <= i+c
```

```
  /\ forall x. x<>m -> 2 * nbc x t i <= i-c
```

Preuve de programme en Why3 : programme annoté

Logique de Hoare (voir cours de GLA)

```
let majority (t : array int) : int
requires {length t <> 0}
ensures {forall x. x <> result -> not (maj x t)}
=
let rec fmaj (i : int) (c : int) (m : int) : int
  requires {inv t i c m}
  ensures {forall x. x <> result -> not (maj x t)}
  variant {length t - i}
  =
    if i = length t then m
    else let x = t[i] in
      if c = 0 || m = x then fmaj (i+1) (c+1) x
      else fmaj (i+1) (c-1) m
in fmaj 0 0 t[0]
```

Preuve automatique en utilisant alt-ergo (*SMT-solver*)



That's all Folks!

1—Maîtriser le langage logique

1 Définition du langage

1—Maîtriser le langage logique

- 1 Définition du langage
 - Objets
 - Formules
 - Traduire des énoncés

- On utilise un langage *formel* pour écrire les énoncés logiques
- Éviter les *ambiguïtés* du langage naturel et les notations imprécises.
 - Je peux t'offrir de l'eau *ou* du vin/Je peux t'offrir de l'eau *et* du vin
 - Le menu propose fromage *ou* dessert/Le menu propose fromage *et* dessert
 - S'il ne pleut pas, je sors jouer/S'il pleut, je ne sors pas jouer
- Moins de *redondance* que le langage naturel : plus simple à étudier, plus simple à implémenter
- Un énoncé écrit dans le langage de la logique sera appelé *formule*

1—Maîtriser le langage logique

- 1 Définition du langage
 - Objets
 - Formules
 - Traduire des énoncés

- Les énoncés parlent d'*objets*
 - entiers, individus, livres, ensembles, fonctions ...
- Dans le langage de la logique, un objet est représenté par un **terme**
- Un terme peut être une constante $0, 1, \text{Martin}, \emptyset, \mathbb{N} \dots$,
- Un terme peut être construit à partir d'*opérations* $+, \times, \cup, \dots$
 $3 + 5, \mathbb{N} \times \mathbb{N}, \text{le père de Martin}, \dots$
Une opération est un *symbole* associé à une **arité**, entier naturel qui représente le nombre d'arguments attendus.
- Dans une formule, on utilise des **variables** : symboles qui représentent des objets indéterminés
 $x + 1, \text{un étudiant de licence}, \dots$

Definition (Signature, arité)

Une signature est un ensemble de symboles $\mathcal{F}$ chacun associé à un entier naturel appelé **arité**.

Un symbole d'arité 0 est appelé **constante**, un symbole d'arité 1 est dit **unaire**, un symbole d'arité 2 est dit **binaire**.

Definition (Terme)

Etant donné une signature $\mathcal{F}$ et un ensemble $\mathcal{X}$ de variables, un terme t est soit une variable, soit formé d'un symbole f d'arité n et d'une suite ordonnée de n termes $t_1, \dots, t_n$.

- f est le **symbole de tête**, $t_1, \dots, t_n$ sont les **sous-termes** directs du terme t .
- $\mathcal{T}(\mathcal{F}, \mathcal{X})$ est l'ensemble des termes sur la signature $\mathcal{F}$ et l'ensemble des variables $\mathcal{X}$.
- $\mathcal{T}(\mathcal{F})$ est l'ensemble des termes qui ne contiennent pas de variable, appelés aussi **termes clos**.

Exemples de signature

- Entiers naturels :
 - constantes 0 et 1
 - opérations binaires : addition + et la multiplication $\times$
 - notation **infixe** : $(t + u)$, $(t \times u)$
 - termes : $x + 1$, $(0 + 1)$, $(1 + 1) \times (1 + 1 + 1)$,...
- Mots binaires de longueur arbitraire
 - constante ϵ pour représenter le mot vide
 - deux fonctions unaires c_0 et c_1 pour représenter l'ajout d'un 0 ou d'un 1 en tête du mot
 - le mot 1011 est représenté par le terme $c_1(c_0(c_1(c_1(\epsilon))))$.
 - autres opérations possibles : concaténation de deux mots, décalage vers la gauche ou la droite

- 1 Définition du langage
 - Objets
 - **Formules**
 - Traduire des énoncés

Formules atomiques

Les **formules atomiques** ne se décomposent pas en formules plus simples.

- $\top$ (top) : la propriété toujours vraie, tautologie ($0 = 0$)
- $\perp$ (bottom) : la propriété toujours fausse, l'absurde ($0 = 1$)
- **symbole de prédicat** associé à un ou plusieurs termes (suivant l'**arité**)
propriétés de base des objets : ce qui ne s'explique pas en terme logique mais qui *s'observe*
 - Exemples de symboles
 - arité 0 (**variable propositionnelle**) : "signal-passage-à-niveau-ouvert",
 - arité 1 (**symbole unaire**, ensemble) : "yeux-bleus", "pair"
 - arité 2 (**symbole binaire**) : l'égalité $=$, la comparaison $\leq$, l'appartenance $\in$,
 - arité quelconque : table d'une base de données
 - Exemples de formules atomiques
 - $0 = 1, 2 + 2 = 4, x = x,$
 - $x + 1 \leq x$
 - *Martin a les yeux bleus* : yeux-bleus(Martin)
 - Etudiant(Durand, Bob, 347890, 01/01/1990)

Exemples de signature : systèmes d'information

- modélisation logique des entités/ensembles et des tables/relation par des symboles de prédicat.
- Trajets de bus :
 - identifiants de ligne (numéro) des arrêts et des horaires : trois prédicats unaires `ligne`, `arrêt` et `horaire` pour séparer les objets de la logique suivant leur catégorie.
`ligne(91-06), arrêt(Massy), arrêt(Saclay), horaire(06h00)...`
 - table qui tient à jour les rotations de bus avec le numéro de ligne, l'arrêt de départ, celui d'arrivée ainsi que l'horaire de départ : predicat `trajet` d'arité 4.
`trajet(91-06, Massy, Saclay, 06h00)`

Choix des symboles de prédicat

- Le choix des symboles de prédicat dépend de la modélisation
 - primitive de base versus notions dérivées via une formule
 - analogie avec les variables d'un problème mathématique ou physique
 - chaque symbole peut s'interpréter librement
 - on peut aussi parfois choisir des symboles de fonction au lieu de symboles de prédicat (notions différentes en logique).
- Exemples
 - *tables* primitives dans une base de données, versus résultat d'une requête
 - *interface* d'une bibliothèque dont on ne connaît pas l'implémentation
 - *axiomatisation* d'une *théorie*
 - entiers : $x \leq y$ versus $\exists n, y = x + n$
 - ordres : $x = y$ versus $(x \leq y \wedge y \leq x)$
 - hommes et femmes, versus $H(x) \stackrel{\text{def}}{=} \neg F(x)$
 - *est-mère*(x, y) versus *x=mère*(y)

- Un symbole est juste un nom (**syntaxe**),
- la **sémantique** lui attribue un *sens* en lui associant une relation mathématique entre les objets modélisés
- il y a parfois un sens usuel implicite (ex : ordre sur les entiers) mais d'un point de vue logique, *toutes les interprétations sont possibles*.
- les sens possibles des symboles seront restreints par l'introduction de **théories**

Formules complexes

Les *formules complexes* (celles qui ne sont pas atomiques) se construisent à l'aide de **connecteurs** et de **quantificateurs** logiques.

Partie **propositionnelle** (connecteurs) :

- $\neg P$ la **négation** d'une formule, prononcée "**non** P "
- $P \wedge Q$ la **conjonction** de deux formules, prononcée " P **et** Q "
- $P \vee Q$ la **disjonction** de deux formules, prononcée " P **ou** Q "
- $P \Rightarrow Q$ l'**implication** de deux formules, prononcée " P **implique** Q " ou bien "**si** P **alors** Q "

Et les **quantificateurs** du **premier ordre** :

- $\forall x, P$ la **quantification universelle**, prononcée "**pour tout** x , P "
- $\exists x, P$ la **quantification existentielle**, prononcée "**il existe** x **tel que** P "

Une formule sans quantificateur $\forall$ et $\exists$ est dite **formule propositionnelle**

Sens intuitif des connecteurs et quantificateurs

- $\neg P$: P est faux
- $P \wedge Q$: P et Q sont tous les deux vrais
- $P \vee Q$: soit P soit Q est vrai (ou les deux)
- $P \Rightarrow Q$: si P est vrai alors Q est vrai et si P est faux alors Q peut être vrai ou faux (P est faux ou bien Q est vrai)
- $\forall x, P$: P est vrai pour toutes les *valeurs* possibles de x
- $\exists x, P$: il existe au moins une *valeur* de x pour laquelle P est vrai

- expressions pour représenter des conditions booléennes
- opérations pour la négation, la conjonction, la disjonction
- pas d'implication : on trouve à la place une conditionnelle

si *a* alors *b* sinon *c*

- *b* et *c* peuvent être des booléens ou représenter d'autres types d'objet
- les quantificateurs ne correspondent pas à des constructions de programme car en général (cas infini) ils ne sont pas *calculables*.

Exercice

On suppose que a , b et c sont des formules logiques.

Représenter la phrase *si a alors b sinon c* comme une formule logique n'utilisant que les connecteurs logiques

- faire la table de vérité
- donner une première représentation sans utiliser d'implication
- donner une seconde représentation qui contienne la formule $a \Rightarrow b$

Exemples de formules complexes

- tiers exclu : $A \vee \neg A$
- modus-ponens : $((A \Rightarrow B) \wedge A) \Rightarrow B$
- loi de Peirce : $((A \Rightarrow B) \Rightarrow A) \Rightarrow A, ((\neg A) \Rightarrow A) \Rightarrow A$
- $\forall x, \text{yeux-bleus}(x)$
- $\exists x, \text{yeux-bleus}(x)$
- $\exists x, (\text{yeux-bleus}(x) \Rightarrow \forall y, \text{yeux-bleus}(y))$
- formule *paramétrée* : l'entier x est impair : $\exists y, x = 2 \times y + 1$

Différentes catégories syntaxiques : termes

- variables (objets) : $\mathcal{X}$
- symboles de fonctions
 - constantes d'arité 0 : $\mathcal{C}$
 - fonctions d'arité au moins 1 : $\mathcal{F}$
- **termes** (ou objets)

$\text{term} \coloneqq \mathcal{X} \mid \mathcal{C} \mid \mathcal{F}(\text{list-terms})$

$\text{list-terms} \coloneqq \text{term} \mid \text{list-terms}, \text{term}$

Différentes catégories syntaxiques : formules

- symboles de prédicats
 - d'arité 0 : $\mathcal{V}$
 - d'arité au moins 1 : $\mathcal{P}$
- **formules** logiques :

$$\begin{aligned} \text{form} \coloneqq & \top \mid \perp \mid \mathcal{V} \mid \mathcal{P}(\text{list-terms}) \\ & \mid \neg \text{form} \mid (\text{form} \wedge \text{form}) \mid (\text{form} \vee \text{form}) \mid (\text{form} \Rightarrow \text{form}) \\ & \mid (\forall \mathcal{X}, \text{form}) \mid (\exists \mathcal{X}, \text{form}) \end{aligned}$$

Notations infixes

- Notation standard :
 - Fonctions/Prédicats : $\text{Symbole}(t_1, \dots, t_n)$
- Quelques notations usuelles **infixes** pour des symboles **binaires** $f(t, u)$
 - $t \circ u$
 - exemples : $t + u, t \times u, t = u, t \leq u \dots$
- Extension de la grammaire

$$\begin{aligned}\text{term} &\coloneqq (\text{term } \mathcal{F}_l \text{ term}) \\ \text{form} &\coloneqq (\text{term } \mathcal{P}_l \text{ term})\end{aligned}$$

les parenthèses évitent les ambiguïtés

Notations, règles de parenthésage

- $A \Leftrightarrow B$ est la même chose que $(A \Rightarrow B) \wedge (B \Rightarrow A)$
- plusieurs variables dans un quantificateur, par exemple : $\forall x y, P$
représente la formule $\forall x, \forall y, P$
- attention $\forall x \in A, P$ ne fait pas partie du langage ni $\exists! x, P$
- l'usage de notations infixes rend nécessaire l'ajout de parenthèses dans la syntaxe des formules
 - comment interpréter $P \wedge Q \vee R$?
 - 1 $(P \wedge Q) \vee R$
 - 2 $P \wedge (Q \vee R)$

- On appelle **logique du premier ordre** (ou **calcul des prédicats**) le langage logique défini par une signature (symboles de fonctions et de prédicats) et qui n'utilise que les connecteurs logiques et les quantificateurs sur les variables de termes tels que définis précédemment.
- Le **calcul propositionnel** (aussi appelé **logique propositionnelle**) est un cas particulier dans lequel la signature est réduite à des symboles de prédicats d'arité 0 (appelées **variables propositionnelles**) et dans lequel on n'utilise pas de quantificateurs.
- Il existe d'autres logiques (logique d'ordre supérieur, logiques temporelles, logiques modales. . .)

- 1 Définition du langage
 - Objets
 - Formules
 - Traduire des énoncés

Langage

- $\text{ami}(x, y)$: x est l'ami de y
- $\text{joue}(x, y)$: x joue avec y
- constante self qui représente l'individu qui s'exprime.

Que signifient les formules suivantes en langage courant ?

- 1 $\forall x, (\text{ami}(\text{self}, x) \Rightarrow \neg \text{joue}(\text{self}, x))$
- 2 $\forall x, \text{joue}(\text{self}, x)$
- 3 $\forall x, \exists y, \text{joue}(x, y)$
- 4 $\forall y, \exists x, \text{joue}(y, x)$
- 5 $\exists y, \forall x, \text{joue}(x, y)$

Exemple de traduction

- le langage comporte un symbole de fonction $+$ binaire noté de manière infixe qui représente l'opération de sommation
- `pair`(x) représente la propriété " x est un entier pair"
- Exprimer par une formule les propriétés :
 - *"la somme de deux entiers pairs est un entier pair"*
 - *"la somme de deux entiers impairs est un entier pair"*

- Connaître les notions de **signature** et **arité**.
- Distinguer les **termes** et les **formules**, et parmi les formules celles qui sont **atomiques**.
- Reconnaître les **termes** et les **formules** *syntactiquement* bien formés.
- Comprendre le sens intuitif des connecteurs propositionnels et quantificateurs logiques.
- Lire une formule logique et traduire une propriété exprimée en langue naturelle en utilisant le langage de la logique.



That's all Folks!